



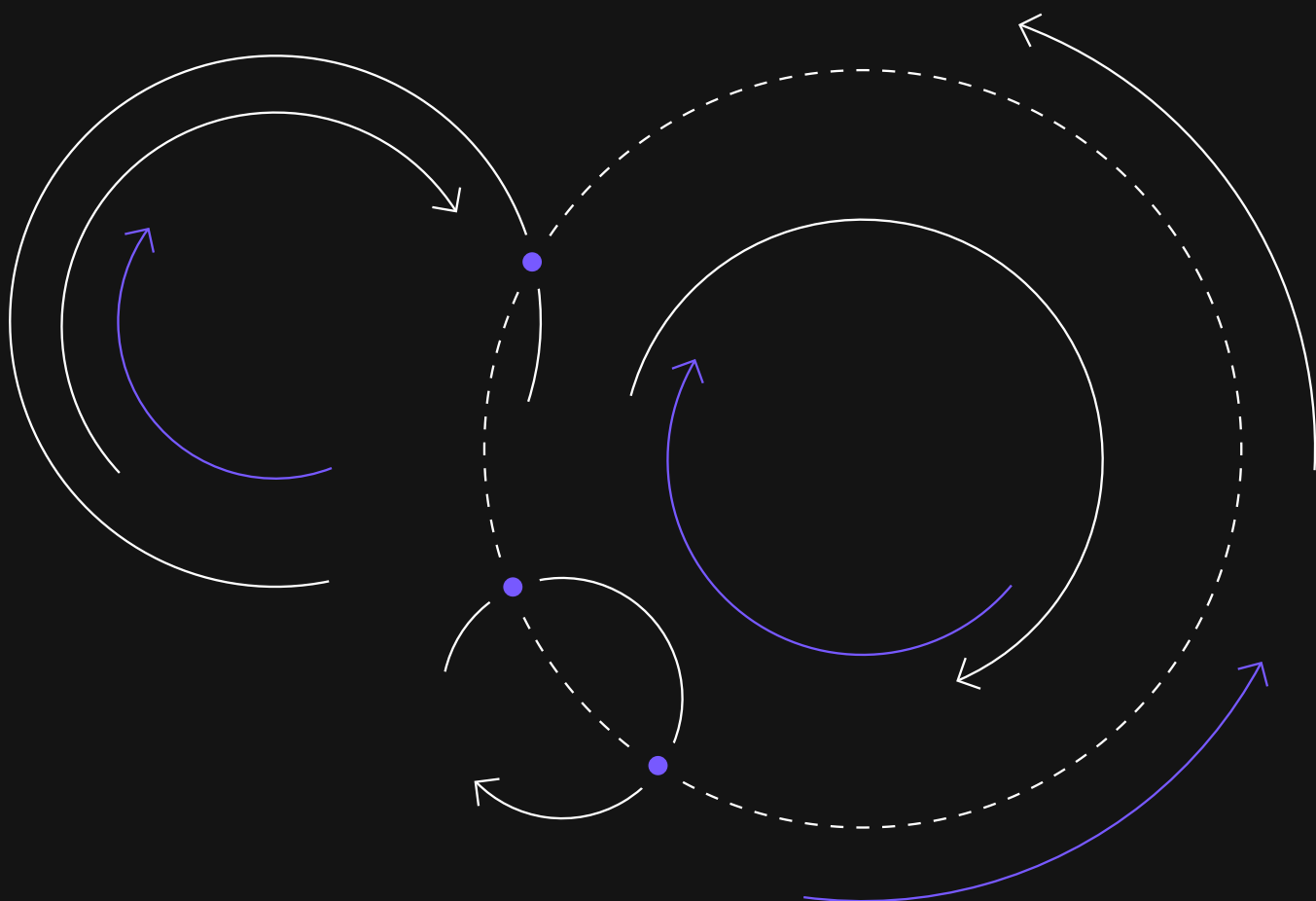
Машинное обучение

Лонгрид

Глубокое обучение

01

Backpropagation, autograd и PyTorch



Содержание

1. От прямого прохода к backpropagation	4
1.1. Полносвязная сеть как граф вычислений	4
1.2. Почему удобно работать с дифференциалами	5
1.3. Конечные разности для проверки градиента	6
1.4. Пример графа вычислений	6
1.5. Три режима автоматического дифференцирования	7
1.5.1. Проход вперёд	7
1.5.2. Проход назад	7
1.5.3. Смешанный режим	8
2. Обратный проход через слой MLP	8
2.1. Покомпонентная функция активации	8
2.1.1. Пример: sigmoid	9
2.1.2. Пример: logsoftmax	9
2.1.3. Softmax и функция потерь на последнем слое	10
2.2. Аффинное преобразование	10
2.3. Проверка размерностей	12
2.4. Стоимость обратного прохода	12
3. Дифференцирование через решение системы $Ax = b$	13
3.1. Дифференциал решения	13
3.2. Вычислительная интерпретация	15
4. Концептуальная модель PyTorch	15
4.1. Вычисления на ускорителе	15
4.2. Автоматическое дифференцирование	16
4.3. Управление памятью	16
5. Тензоры, формы и память	17
5.1. NumPy и PyTorch	17
5.2. Изменение формы	18
5.3. Broadcasting, view и reshape	18
5.3.1. Иногда broadcasting — это зло	19
5.4. In-place-операции	19
5.5. Устройство и тип данных	20
6. Как работает autograd	21
6.1. requires_grad и backward	21
6.2. Forward mode и JVP	21
6.3. clone и detach	22
6.4. Градиенты накапливаются	23
6.5. Листовые и промежуточные тензоры	24
6.6. Жизненный цикл графа	24
6.7. Обучение и инференс — разные режимы	25
7. Dataset и DataLoader	25

7.1. Dataset	25
7.2. DataLoader	26
7.3. Преобразования данных	27
8. Модель как nn.Module	27
8.1. Иерархия блоков	28
8.2. Предобучение и новая «голова»	29
8.3. Число параметров	29
9. Оптимизатор и цикл обучения	30
9.1. Одна эпоха обучения	30
10. Оценка модели	31
11. Основные выводы	32

Лекция 2. Backpropagation, autograd и PyTorch.

Матричный обратный проход и базовый цикл обучения

О чём эта лекция

В этой лекции мы выведем матричные формулы обратного прохода для полно-связного слоя, продифференцируем решение системы линейных уравнений, а затем разберём основные механизмы PyTorch: динамический вычислительный граф, autograd, размещение тензоров на устройствах, работу с памятью, Dataset, DataLoader, nn.Module и стандартные циклы обучения и оценки модели.

1. От прямого прохода к backpropagation

1.1. ПОЛНОСВЯЗНАЯ СЕТЬ КАК ГРАФ ВЫЧИСЛЕНИЙ

Каждый скрытый слой многослойного персептрона выполняет два преобразования:

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \varphi_{\ell}(z^{(\ell)}), \quad \ell = 1, \dots, L-1,$$

где φ_{ℓ} применяется к каждой компоненте вектора отдельно. Начальная активация равна входу: $a^{(0)} = x$. После последнего слоя вычисляется скалярная функция потерь $\mathcal{L}$.

Как и в первой лекции, в задаче классификации последний слой удобно рассматривать отдельно:

$$z^{(L)} = W^{(L)}a^{(L-1)} + b^{(L)}, \quad p = \text{softmax}(z^{(L)}), \quad \mathcal{L} = -\log p_y.$$

Ниже сначала выводится правило для произвольного скрытого слоя, а градиент последнего слоя считается из выбранной функции потерь.

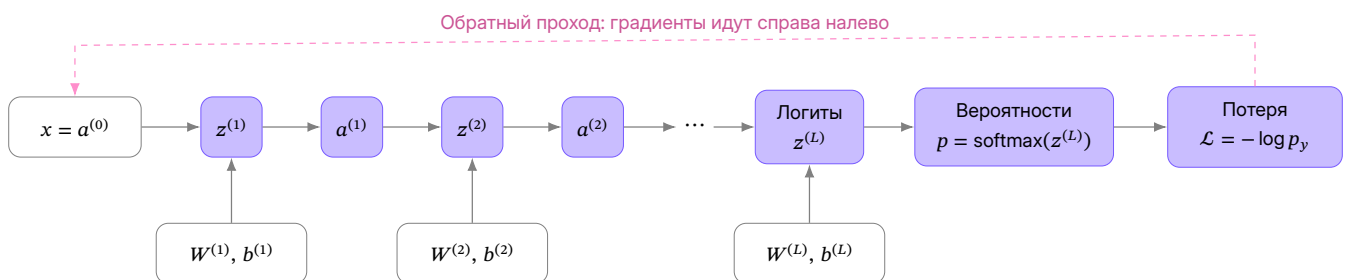


Рисунок 1. Прямой проход вычисляет вершины графа слева направо и сохраняет значения, нужные для производных; на обратном проходе каждому слою передаётся градиент по его входу, а внутри слоя вычисляются градиенты по $W^{(\ell)}$ и $b^{(\ell)}$

Во время **прямого прохода** (*forward pass*) сеть последовательно вычисляет вершины графа и сохраняет значения, которые понадобятся для производных. Во время **обратного прохода** (*backward pass*) градиент распространяется от $\mathcal{L}$ справа налево.

Для обучения нужны градиенты по всем параметрам:

$$\nabla_{W^{(\ell)}} \mathcal{L}, \quad \nabla_{b^{(\ell)}} \mathcal{L}, \quad \ell = 1, \dots, L.$$

**Основные обозначения**

- $x = a^{(0)}$ — входной вектор признаков;
- L — число обучаемых слоёв, n_ℓ — ширина ℓ -го слоя;
- $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ и $b^{(\ell)} \in \mathbb{R}^{n_\ell}$ — веса и смещения слоя;
- $z^{(\ell)}$ — значения до функции активации, $a^{(\ell)}$ — активации скрытого слоя, $\ell = 1, \dots, L-1$;
- $z^{(L)}$ — логиты последнего слоя, $p = \text{softmax}(z^{(L)})$ — вероятности классов;
- e_y — one-hot-вектор правильного класса y ;
- φ_ℓ — покомпонентная функция активации ℓ -го слоя;
- $\mathcal{L}$ — скалярная функция потерь; матричный вывод в этом разделе проводится для одного объекта, а пакетная запись приводится отдельно;
- $\nabla_u \mathcal{L}$ — градиент функции потерь по u , записан как вектор-столбец;
- $\delta^{(\ell)} := \nabla_{z^{(\ell)}} \mathcal{L}$ — градиент по входу нелинейности ℓ -го слоя;
- $\odot$ — покомпонентное произведение, dX — дифференциал переменной X .

1.2. ПОЧЕМУ УДОБНО РАБОТАТЬ С ДИФФЕРЕНЦИАЛАМИ

Покомпонентное правило цепочки быстро превращается в громоздкую систему индексов. В матричной записи удобнее:

- 1) вычислить дифференциал функции,
- 2) привести его к канонической форме,
- 3) прочесть из полученного выражения градиент.

Вспомним базовые определения на примере функции

$$f(x) = \frac{1}{2} x^\top A x, \quad A \succ 0.$$

Дифференциалом мы называем линейную часть приращения функции f :

$$df = Df(x)[dx] = (dx)^\top A x.$$

Производной Фреше мы называем линейный оператор, результатом действия которого является дифференциал:

$$Df(x)[\cdot] = \langle A x, \cdot \rangle.$$

Для скалярной функции производную Фреше задаёт градиент:

$$\frac{\partial f}{\partial x} = \nabla_x f = A x.$$

Для векторнозначной функции аналогичную роль играет матрица Якоби.

Вспомним важное свойство — **инвариантность первой формы дифференциала**.

Если $y = g(z)$, $z = f(x)$, то

$$dy = \left\langle \frac{\partial y}{\partial z}, dz \right\rangle = \left\langle \frac{\partial y}{\partial x}, dx \right\rangle.$$

Для скалярной функции векторного аргумента $f : \mathbb{R}^n \rightarrow \mathbb{R}$

$$df = (\nabla_x f)^\top dx.$$

Для скалярной функции матричного аргумента $f : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$

$$df = \text{tr}((\nabla_X f)^\top dX).$$

Один и тот же скаляр $d\mathcal{L}$ можно записать через разные промежуточные переменные графа.

Сравнение канонических форм и даёт правила обратного прохода.

1.3. КОНЕЧНЫЕ РАЗНОСТИ ДЛЯ ПРОВЕРКИ ГРАДИЕНТА

Помимо автоматического дифференцирования, есть другой подход к подсчёту градиента — конечные разности. Пусть есть функция $f(x)$. Производную по направлению d можно приблизить так:

$$\nabla f(x)^\top d \approx \frac{f(x + \varepsilon d) - f(x)}{\varepsilon}.$$

Данный способ хорошо подходит для проверки корректности подсчёта градиента, но не пригоден для использования во время обучения. Вместо разностного подхода используют методы **автоматического дифференцирования** — вычисление производной по графу вычислений.

1.4. ПРИМЕР ГРАФА ВЫЧИСЛЕНИЙ

Рассмотрим функцию

$$f(x_1, x_2, x_3) = \frac{x_1 x_2 \sin(x_3) + \exp(x_1 x_2)}{x_3}.$$

Один из графов, реализующих эту функцию, можно записать так.

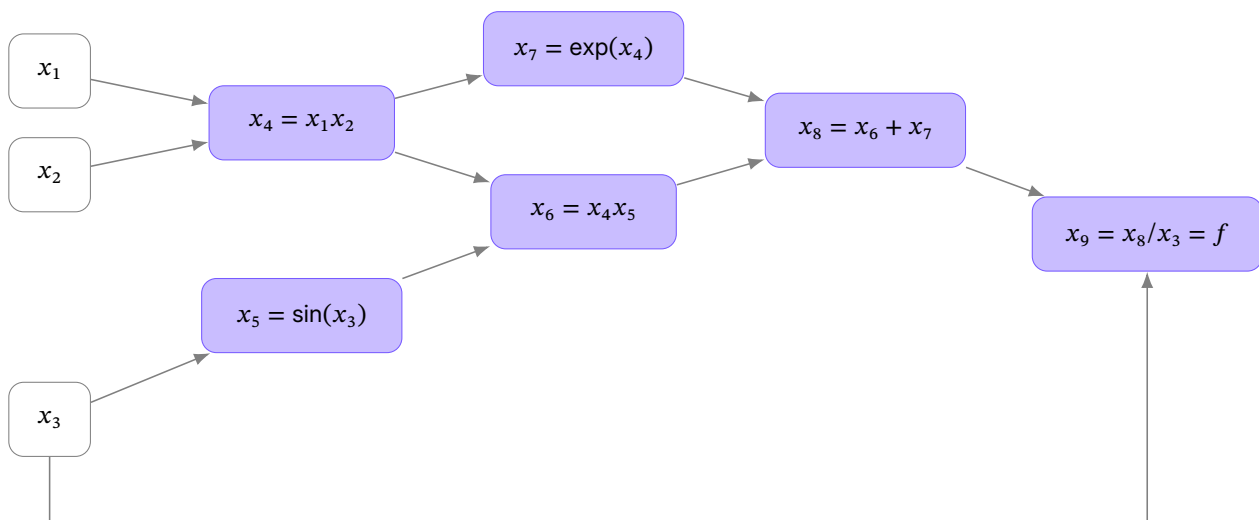


Рисунок 2. Граф вычислений для $f(x_1, x_2, x_3)$: входные переменные слева, промежуточные вершины в середине, результат справа

Граф вычислений не задаётся формулой однозначно. Поскольку функцию мы вычисляем на Python, конкретный граф строится по выполненному коду. Основная цель — получить производные f по независимым переменным x_1, x_2, x_3 .

1.5. ТРИ РЕЖИМА АВТОМАТИЧЕСКОГО ДИФФЕРЕНЦИРОВАНИЯ

В общем случае выделяют три режима:

- вычисление производной проходом вперёд (*forward mode*);
- вычисление производной проходом назад (*backward mode*);
- смешанный режим.

1.5.1. Проход вперёд

Производные вычисляются слева направо:

$$\frac{\partial x_i}{\partial x_j}, \quad i = 1, \dots, 9, \quad j = 1, 2, 3.$$

Например, для производных по x_1 начальные значения равны:

$$\frac{\partial x_1}{\partial x_1} = 1, \quad \frac{\partial x_2}{\partial x_1} = 0, \quad \frac{\partial x_3}{\partial x_1} = 0.$$

Далее вместе со значениями вершин пересчитываются их производные:

$$\begin{aligned} x_4 &= x_1 x_2, & \frac{\partial x_4}{\partial x_1} &= \frac{\partial x_1}{\partial x_1} x_2 + x_1 \frac{\partial x_2}{\partial x_1}, \\ x_5 &= \sin(x_3), & \frac{\partial x_5}{\partial x_1} &= \cos(x_3) \frac{\partial x_3}{\partial x_1}. \end{aligned}$$

Для произвольной вершины графа получаем

$$\frac{\partial x_j}{\partial x_i} = \sum_{k: (k,j) \in \Sigma} \frac{\partial x_j}{\partial x_k} \frac{\partial x_k}{\partial x_i},$$

где $(k, j) \in \Sigma$ означает, что в графе есть ребро из k в j . Производные достаточно хранить для предыдущего слоя графа: после перехода на новый слой более ранние значения можно забыть.

1.5.2. Проход назад

Теперь производные вычисляются справа налево:

$$\frac{\partial f}{\partial x_j}, \quad j = 1, \dots, 9.$$

Начальное значение:

$$\frac{\partial f}{\partial x_9} = 1.$$

Далее идём в обратном направлении. Например,

$$\frac{\partial f}{\partial x_8} = \frac{\partial f}{\partial x_9} \frac{\partial x_9}{\partial x_8} = \frac{\partial f}{\partial x_9} \frac{1}{x_3},$$

а для вершины x_4 , от которой выходят два ребра:

$$\frac{\partial f}{\partial x_4} = \frac{\partial f}{\partial x_6} \frac{\partial x_6}{\partial x_4} + \frac{\partial f}{\partial x_7} \frac{\partial x_7}{\partial x_4}.$$

В общем случае

$$\frac{\partial f}{\partial x_j} = \sum_{k: (j,k) \in \Sigma} \frac{\partial f}{\partial x_k} \frac{\partial x_k}{\partial x_j}.$$

После одного обратного прохода получаем полный вектор градиента.

1.5.3. Смешанный режим

Пусть в графе есть скалярная промежуточная переменная y , а нас интересует матрица Якоби $\partial f_j / \partial x_i$. Можно использовать структуру графа: левую часть пройти назад, а правую — вперёд. Тогда

$$\frac{\partial f_j}{\partial x_i} = \frac{\partial f_j}{\partial y} \frac{\partial y}{\partial x_i}.$$

Такой подход основан на анализе графа вычислений и поиске подграфов, которые можно вычислить эффективнее. Эта задача похожа на работу компилятора и применяемые в нём оптимизации.

На практике прямой и смешанный режимы используются реже: основные фреймворки прежде всего реализуют обратный проход. К прямому режиму в PyTorch мы вернёмся в [разделе 6.2](#).

2. Обратный проход через слой MLP

2.1. ПОКОМПОНЕНТНАЯ ФУНКЦИЯ АКТИВАЦИИ

Пусть градиент $\nabla_{a^{(\ell)}} \mathcal{L}$ уже известен. Нужно перенести его через преобразование

$$a^{(\ell)} = \varphi_\ell(z^{(\ell)}).$$

Якобиан покомпонентной функции диагонален:

$$da^{(\ell)} = \text{diag}(\varphi'_\ell(z^{(\ell)})) dz^{(\ell)}.$$

Подставим это равенство в дифференциал функции потерь:

$$\begin{aligned} d\mathcal{L} &= (\nabla_{a^{(\ell)}} \mathcal{L})^\top da^{(\ell)} = \\ &= (\nabla_{a^{(\ell)}} \mathcal{L})^\top \text{diag}(\varphi'_\ell(z^{(\ell)})) dz^{(\ell)}. \end{aligned}$$

Следовательно,

$$\boxed{\nabla_{z^{(\ell)}} \mathcal{L} = \nabla_{a^{(\ell)}} \mathcal{L} \odot \varphi'_\ell(z^{(\ell)})}.$$

То есть при проходе через покомпонентную активацию входящий градиент умножается на производную активации покомпонентно.

2.1.1. Пример: sigmoid

Пусть

$$a = f(z) = \frac{1}{1 + \exp(-z)},$$

где $f : \mathbb{R} \rightarrow \mathbb{R}$ применяется поэлементно, а $z \in \mathbb{R}^{\dots}$ — произвольный тензор.

Найдём $\nabla_z \mathcal{L}$ при известном $\nabla_a \mathcal{L} \in \mathbb{R}^{\dots}$. Дифференциал:

$$\begin{aligned} da &= f(z)(1 - f(z)) \odot dz = \\ &= a(1 - a) \odot dz. \end{aligned}$$

По свойству инвариантности первого вида дифференциала

$$\begin{aligned} d\mathcal{L} &= \langle \nabla_a \mathcal{L}, da \rangle = \\ &= \left\langle \underbrace{(\nabla_a \mathcal{L}) \odot a \odot (1 - a)}_{\nabla_z \mathcal{L}}, dz \right\rangle. \end{aligned}$$

Следовательно,

$$\boxed{\nabla_z \mathcal{L} = (\nabla_a \mathcal{L}) \odot a \odot (1 - a)}.$$

2.1.2. Пример: logsoftmax

Пусть

$$a = \text{logsoftmax}(z), \quad a_j = \log \frac{\exp(z_j)}{\sum_i \exp(z_i)},$$

где $a, z \in \mathbb{R}^K$, а логарифм и экспонента применяются покомпонентно. Введём

$$p = \text{softmax}(z), \quad p_i = \frac{\exp(z_i)}{\sum_k \exp(z_k)}.$$

Тогда

$$da_j = dz_j - \sum_i p_i dz_i.$$

Поэтому

$$\begin{aligned} d\mathcal{L} &= \sum_j \frac{\partial \mathcal{L}}{\partial a_j} da_j = \\ &= \sum_i \left(\frac{\partial \mathcal{L}}{\partial a_i} - p_i \sum_j \frac{\partial \mathcal{L}}{\partial a_j} \right) dz_i. \end{aligned}$$

Следовательно,

$$\boxed{\nabla_z \mathcal{L} = \nabla_a \mathcal{L} - p(\mathbf{1}^\top \nabla_a \mathcal{L})}.$$



Не перепутай p и a

Коэффициенты при dz_i в этом выводе — вероятности $p_i = \text{softmax}(z)_i$, а не значения $a_i = \text{logsoftmax}(z)_i$.

2.1.3. Softmax и функция потерь на последнем слое

Рассмотрим последний слой многоклассовой классификации. Пусть $a^{(L-1)} \in \mathbb{R}^d$, $y \in \{1, 2, \dots, K\}$:

$$z^{(L)} = W^{(L)}a^{(L-1)} + b^{(L)}, \quad W^{(L)} \in \mathbb{R}^{K \times d}.$$

Для компактности дальнейшего вывода положим $z = z^{(L)}$. Вероятности классов обозначим через

$$p = \frac{\exp(z)}{\exp(z)^T \mathbf{1}},$$

где $\exp(z)$ — покомпонентная функция, а $\mathbf{1}$ — вектор из единиц. Функция потерь записывается как

$$\mathcal{L}(p, y) = -\log p^T e_y.$$

Для её дифференциала выполнено:

$$d\mathcal{L} = \nabla_p \mathcal{L}^T dp = -\frac{1}{p^T e_y} e_y^T dp.$$

Пользуясь инвариантностью первого дифференциала, запишем

$$d\mathcal{L} = \nabla_p \mathcal{L}^T dp = \nabla_z \mathcal{L}^T dz.$$

Продифференцируем p по определению:

$$\begin{aligned} dp &= \frac{\text{diag}(\exp(z)) dz \exp(z)^T \mathbf{1} - \exp(z) \mathbf{1}^T \text{diag}(\exp(z)) dz}{(\exp(z)^T \mathbf{1})^2} = \\ &= \frac{1}{\exp(z)^T \mathbf{1}} \text{diag}(\exp(z)) dz - \frac{1}{(\exp(z)^T \mathbf{1})^2} \exp(z) \mathbf{1}^T \text{diag}(\exp(z)) dz. \end{aligned}$$

После подстановки dp в $d\mathcal{L}$ получаем

$$\nabla_z \mathcal{L} = \frac{1}{\exp(z)^T \mathbf{1}} \text{diag}(\exp(z)) \nabla_p \mathcal{L} - \frac{\nabla_p \mathcal{L}^T \exp(z)}{(\exp(z)^T \mathbf{1})^2} \text{diag}(\exp(z)) \mathbf{1}.$$

Для кросс-энтропии с правильным классом y это выражение упрощается до

$$\delta^{(L)} = \nabla_{z^{(L)}} \mathcal{L} = p - e_y.$$

Дальше градиенты по $W^{(L)}$, $a^{(L-1)}$ и $b^{(L)}$ находятся через дифференциал аффинного преобразования.

2.2. АФФИННОЕ ПРЕОБРАЗОВАНИЕ

Теперь перенесём градиент через

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}.$$

Для краткости введём обозначение

$$\delta^{(\ell)} := \nabla_{z^{(\ell)}} \mathcal{L}.$$

Дифференциал выхода слоя равен

$$dz^{(\ell)} = dW^{(\ell)}a^{(\ell-1)} + W^{(\ell)}da^{(\ell-1)} + db^{(\ell)}.$$

Тогда

$$d\mathcal{L} = (\delta^{(\ell)})^\top dW^{(\ell)} a^{(\ell-1)} + \\ + (\delta^{(\ell)})^\top W^{(\ell)} da^{(\ell-1)} + \\ + (\delta^{(\ell)})^\top db^{(\ell)}.$$

Для первого слагаемого применим циклическое свойство следа:

$$(\delta^{(\ell)})^\top dW^{(\ell)} a^{(\ell-1)} = \text{tr}((\delta^{(\ell)})^\top dW^{(\ell)} a^{(\ell-1)}) = \\ = \text{tr}\left(\left[\delta^{(\ell)}(a^{(\ell-1)})^\top\right]^\top dW^{(\ell)}\right).$$

Отсюда получаем три backward-формулы:

$$\boxed{\nabla_{W^{(\ell)}} \mathcal{L} = \delta^{(\ell)} (a^{(\ell-1)})^\top}, \quad \boxed{\nabla_{b^{(\ell)}} \mathcal{L} = \delta^{(\ell)}}, \quad \boxed{\nabla_{a^{(\ell-1)}} \mathcal{L} = (W^{(\ell)})^\top \delta^{(\ell)}}.$$



Как читать эти формулы

- Градиент по весам — внешнее произведение входящего градиента и активаций предыдущего слоя.
- Градиент по смещению совпадает с градиентом по $z^{(\ell)}$.
- Для перехода к предыдущему слою градиент умножается на транспонированную матрицу весов.

Прямой проход

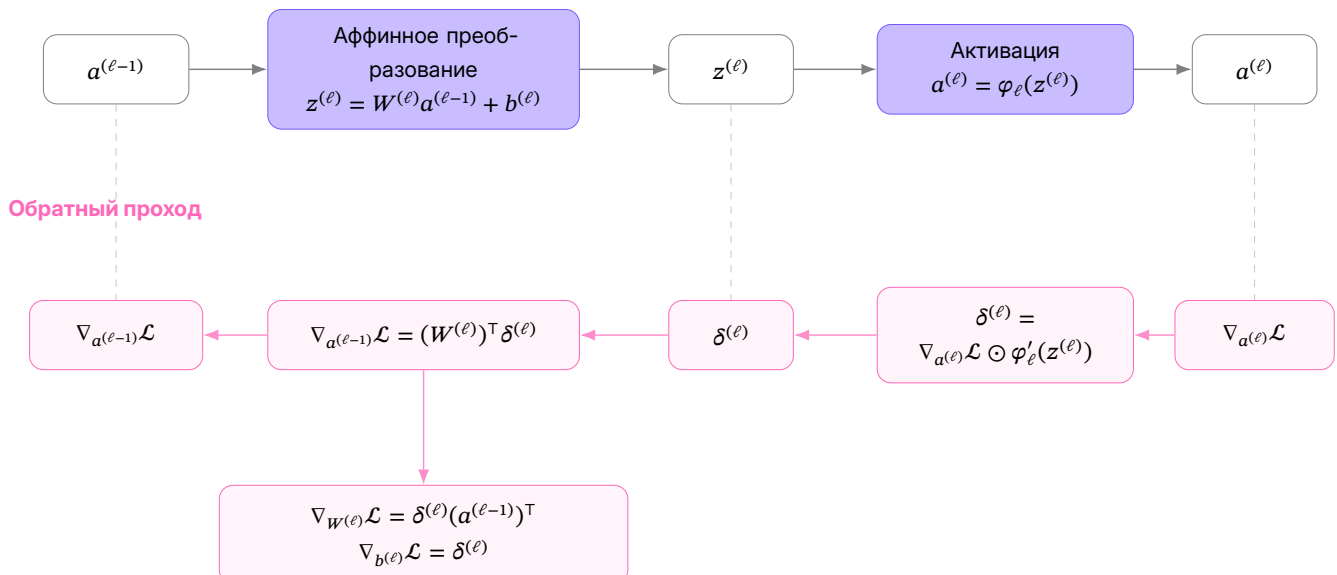


Рисунок 3. Прямой проход слоя (сверху) и обратный проход (снизу): каждой величине графа отвечает градиент в той же позиции, а из аффинного блока дополнительно выходят градиенты по параметрам

2.3. ПРОВЕРКА РАЗМЕРНОСТЕЙ

Величина	Размерность
$a^{(\ell-1)}$	$n_{\ell-1} \times 1$
$\delta^{(\ell)}$	$n_{\ell} \times 1$
$W^{(\ell)}$	$n_{\ell} \times n_{\ell-1}$
$\nabla_{W^{(\ell)}} \mathcal{L}$	$n_{\ell} \times n_{\ell-1}$
$\nabla_{b^{(\ell)}} \mathcal{L}$	$n_{\ell} \times 1$
$\nabla_{a^{(\ell-1)}} \mathcal{L}$	$n_{\ell-1} \times 1$

Таблица 1. Размерности величин, участвующих в обратном проходе слоя

В матричном выводе для батча из B объектов соберём активации и градиенты отдельных объектов в столбцы:

$$A^{(\ell-1)} \in \mathbb{R}^{n_{\ell-1} \times B}, \quad \Delta^{(\ell)} = \begin{bmatrix} \delta_1^{(\ell)} & \dots & \delta_B^{(\ell)} \end{bmatrix} \in \mathbb{R}^{n_{\ell} \times B}.$$

Тогда для суммы потерь по батчу

$$\nabla_{W^{(\ell)}} \mathcal{L} = \Delta^{(\ell)} (A^{(\ell-1)})^T, \quad \nabla_{b^{(\ell)}} \mathcal{L} = \Delta^{(\ell)} \mathbf{1}_B.$$

Если функция потерь усредняется по объектам, обе суммы делятся на B .



Соглашение о размере батча

В формулах выше объекты батча стоят в **столбцах**: $A^{(\ell-1)} \in \mathbb{R}^{n_{\ell-1} \times B}$.

В коде PyTorch обычно используется batch-first-запись, поэтому тот же набор активаций имеет форму $(B, n_{\ell-1})$.

2.4. СТОИМОСТЬ ОБРАТНОГО ПРОХОДА

Обратный проход обычно имеет тот же порядок вычислительной сложности, что и прямой:

- умножение $W^{(\ell)} a^{(\ell-1)}$ заменяется умножением на $(W^{(\ell)})^T$;
- покомпонентная функция заменяется её покомпонентной производной;
- для градиента по весам вычисляется внешнее произведение.

Это одна из ключевых причин практической ценности автоматического дифференцирования: градиент по огромному числу параметров не требует отдельного запуска функции для каждого параметра.

3. Дифференцирование через решение системы $Ax = b$

Backpropagation применяется не только к стандартным нейросетевым слоям. Рассмотрим вычислительную операцию

$$Ax = b,$$

где $A \in \mathbb{R}^{n \times n}$ — невырожденная матрица, $b \in \mathbb{R}^n$, а найденный вектор x затем используется для вычисления $\mathcal{L}(x)$.



Локальные обозначения

В этом разделе A , b и x относятся только к операции решения линейной системы. Здесь x — найденное решение, а не вход MLP из предыдущих разделов.

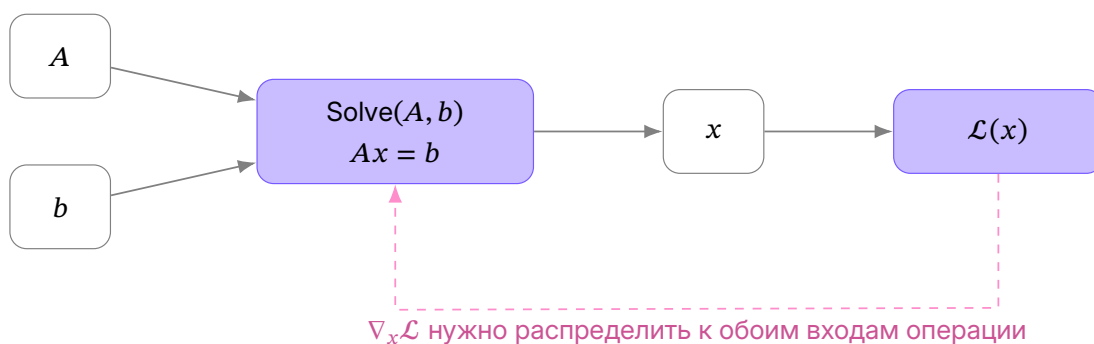


Рисунок 4. Решение линейной системы как вершина вычислительного графа: у операции два входа, на обратном проходе градиент нужно получить и для A , и для b

На обратном проходе известен

$$q := \nabla_x \mathcal{L}.$$

Требуется найти $\nabla_A \mathcal{L}$ и $\nabla_b \mathcal{L}$.

3.1. ДИФФЕРЕНЦИАЛ РЕШЕНИЯ

Продифференцируем исходное равенство, не вычисляя обратную матрицу:

$$(dA)x + A dx = db.$$

Отсюда

$$dx = A^{-1}(db - (dA)x).$$

Подставим это в дифференциал функции потерь:

$$\begin{aligned} d\mathcal{L} &= q^\top dx = \\ &= q^\top A^{-1} db - q^\top A^{-1} (dA)x. \end{aligned}$$

Введём вспомогательный вектор

$$\lambda := A^{-\top} q.$$

Вычислительно это означает не построение $A^{-\top}$, а решение ещё одной системы:

$$A^{\top} \lambda = q.$$

Тогда

$$d\mathcal{L} = \lambda^{\top} db - \lambda^{\top} (dA)x$$

и, следовательно,

$$\boxed{\nabla_b \mathcal{L} = \lambda}, \quad \boxed{\nabla_A \mathcal{L} = -\lambda x^{\top}}.$$

Эквивалентная запись без вспомогательного обозначения:

$$\nabla_b \mathcal{L} = A^{-\top} q, \quad \nabla_A \mathcal{L} = -A^{-\top} q x^{\top}.$$



Эквивалентный вывод через производную обратной матрицы

На доске тот же результат был получен из явной записи $x = A^{-1}b$.

Продифференцируем её:

$$dx = d(A^{-1})b + A^{-1}db.$$

Из тождества $A^{-1}A = I$ следует:

$$d(A^{-1}) = -A^{-1}(dA)A^{-1},$$

поэтому

$$dx = -A^{-1}(dA)A^{-1}b + A^{-1}db.$$

При $q = \nabla_x \mathcal{L}$ получаем

$$\begin{aligned} d\mathcal{L} &= q^{\top} dx = \\ &= -q^{\top} A^{-1}(dA)A^{-1}b + q^{\top} A^{-1}db. \end{aligned}$$

Следовательно,

$$\nabla_A \mathcal{L} = -A^{-\top} q b^{\top} A^{-\top} = -A^{-\top} q x^{\top}, \quad \nabla_b \mathcal{L} = A^{-\top} q.$$

Для реализации всё равно предпочтителен вывод через $Ax = b$: он приводит к решению системы $A^{\top} \lambda = q$ и не требует явно строить A^{-1} .

3.2. ВЫЧИСЛИТЕЛЬНАЯ ИНТЕРПРЕТАЦИЯ

Этап	Что нужно сделать
Forward	Решить $Ax = b$ и получить x
Backward	Решить $A^T \lambda = \nabla_x \mathcal{L}$
Backward	Вычислить $\nabla_A \mathcal{L} = -\lambda x^T$ и $\nabla_b \mathcal{L} = \lambda$

Таблица 2. Что вычисляется на прямом и обратном проходе операции solve



Не вычисляй A^{-1} явно

Формулы с A^{-1} удобны для вывода, но на компьютере систему нужно решать специализированным алгоритмом. Явное обращение матрицы обычно медленнее и менее устойчиво.

Если на прямом проходе была построена LU-факторизация $A = LU$, её можно сохранить и переиспользовать на обратном проходе. Тогда повторная факторизация порядка $O(n^3)$ не нужна: остаются решения треугольных систем порядка $O(n^2)$.

Этот пример показывает, зачем иногда приходится выводить backward вручную. Для стандартных операций PyTorch уже знает производные, но нестандартную дифференцируемую операцию можно оформить как собственную `torch.autograd.Function` с методами `forward` и `backward`.

4. Концептуальная модель PyTorch

PyTorch можно воспринимать как библиотеку многомерных массивов, похожую на NumPy, но с тремя принципиальными дополнениями.

4.1. ВЫЧИСЛЕНИЯ НА УСКОРИТЕЛЕ

Тензоры и модель можно разместить на GPU. Это особенно важно для матричных операций, составляющих основную часть вычислений нейросети.

4.2. АВТОМАТИЧЕСКОЕ ДИФФЕРЕНЦИРОВАНИЕ

Во время прямого прохода PyTorch записывает выполненные операции в вычислительный граф. Вызов `backward()` проходит по графу в обратном направлении и вычисляет необходимые градиенты.

Граф **динамический**: он строится при фактическом выполнении программы. Поэтому его структура может зависеть от входных данных, ветвлений и числа итераций цикла.

Например, если внутри `forward` выполняется итерационный алгоритм до сходимости, число его шагов заранее неизвестно. Динамический граф естественно отражает именно те операции, которые были выполнены в данном запуске.



Статический и динамический граф

В системах со статическим графом, например в Theano и TensorFlow 1.x, граф сначала задавали целиком и компилировали. PyTorch исторически сделал основной сценарий более похожим на обычное выполнение Python: граф строится на ходу. Конкретные библиотеки меняются, поэтому важнее понимать сам принцип, чем запоминать один API.

4.3. УПРАВЛЕНИЕ ПАМЯТЬЮ

Память расходуют не только параметры модели:

- но и мини-батчи данных;
- промежуточные активации, сохранённые для `backward`;
- сами градиенты;
- внутреннее состояние оптимизатора.

GPU-памяти обычно меньше, чем оперативной памяти CPU, поэтому при работе с каждой операцией полезно задавать три вопроса.

1. На каком устройстве находятся тензоры?
2. Являются ли они частью вычислительного графа?
3. Создаёт ли операция новый буфер, представление или изменение на месте?

5. Тензоры, формы и память

5.1. NUMPY И PYTORCH

Интерфейс PyTorch похож на NumPy, но не совпадает с ним полностью.

Действие	NumPy	PyTorch
Создать массив	<code>np.array(data)</code>	<code>torch.tensor(data)</code>
Узнать тип	<code>x.dtype</code>	<code>x.dtype</code>
Узнать форму	<code>x.shape</code>	<code>x.shape, x.size()</code>
Изменить форму	<code>x.reshape(...)</code>	<code>x.reshape(...), x.view(...)</code>
Указать ось редукции	<code>axis=...</code>	<code>dim=...</code>
Матричное умножение	<code>np.matmul(a, b), a @ b</code>	<code>torch.matmul(a, b), a @ b</code>
Изменить тип	<code>x.astype(np.int64)</code>	<code>x.to(torch.int64)</code>
Среднее по оси	<code>x.mean(axis=-1)</code>	<code>x.mean(dim=-1)</code>
Накопленная сумма	<code>x.cumsum(axis=0)</code>	<code>x.cumsum(dim=0)</code>

Таблица 3. Соответствие типичных операций NumPy и PyTorch

Типичные операции почти дословно переносятся из NumPy.

```

1 x = torch.arange(16, dtype=torch.float32).view(4, 4)
2
3 x + 5          # Поэлементное сложение со скаляром
4 x @ x.T        # Матричное умножение
5 x.mean(dim=-1) # Среднее по последней оси
6 x.cumsum(dim=0) # Накопленная сумма вдоль первой оси
7 x.to(torch.int64) # Аналог astype(np.int64)
```

Главное различие в именах аргументов: в NumPy ось обычно задаётся через `axis`, а в PyTorch — через `dim`. Преобразования между CPU-массивом NumPy и CPU-тензором PyTorch часто используют одну и ту же память.

```

1 x = torch.from_numpy(array)
2 array = x.numpy()
```

Если тензор связан с графом или находится на GPU, безопасный шаблон выглядит так.

```
1 array = x.detach().cpu().numpy()
```

Если нужна независимая копия, добавляют `.copy()`.

5.2. ИЗМЕНЕНИЕ ФОРМЫ

`squeeze` удаляет оси размера 1:

$$(2, 1, 2, 1, 2) \xrightarrow{\text{squeeze}} (2, 2, 2).$$

Если передать номер оси, удаляется только она только при условии, что её размер равен 1. `unsqueeze`, наоборот, добавляет новую ось.

```
1 x = torch.zeros(2, 1, 2, 1, 2)
2 print(x.squeeze().shape) # torch.Size([2, 2, 2])
3 print(x.squeeze(1).shape) # torch.Size([2, 2, 1, 2])
4
5 y = torch.zeros(2, 2, 2)
6 print(y.unsqueeze(1).shape) # torch.Size([2, 1, 2, 2])
```

`torch.cat` объединяет тензоры вдоль уже существующей оси.

```
1 m = torch.tensor([[1, 2, 3],
2                  [4, 5, 6]])
3
4 print(torch.cat((m, m, m), dim=0).shape) # torch.Size([6, 3])
5 print(torch.cat((m, m, m), dim=1).shape) # torch.Size([2, 9])
```

Формы должны совпадать по всем осям, кроме оси конкатенации. В отличие от `torch.stack`, операция `torch.cat` не создаёт новую ось.

5.3. BROADCASTING, VIEW И RESHAPE

PyTorch поддерживает broadcasting. Например,

$$A \in \mathbb{R}^{4 \times 1}, \quad b \in \mathbb{R}^{1 \times 4} \implies A + b \in \mathbb{R}^{4 \times 4}.$$

```
1 a = torch.tensor([1.0, 1.0, 1.0, 2.0]).view(4, 1) # (4, 1)
2 b = torch.tensor([1.0, 0.0, 1.0, 0.0]).view(1, 4) # (1, 4)
3
4 print(a + b)
5 # tensor([[2., 1., 2., 1.],
6 #         [2., 1., 2., 1.],
7 #         [2., 1., 2., 1.],
8 #         [3., 2., 3., 2.]])
9
10 c = torch.randn(4, 4)
11 print((b + c).shape) # torch.Size([4, 4])
```

Размер 1 в форме тензора означает, что значения можно распространить вдоль соответствующей оси без явного создания копий.

5.3.1. Иногда broadcasting — это зло

Рассмотрим линейную регрессию.

```

1 x = torch.randn(20, 10)
2 W = torch.randn(10, 1)
3 y = torch.randn(20)
4
5 preds = x @ W
6 print(preds.shape) # torch.Size([20, 1])
7 print(y.shape)     # torch.Size([20])
8
9 diff = preds - y
10 print(diff.shape)  # torch.Size([20, 20])

```

Мы ожидали вектор из 20 ошибок, но из-за broadcasting получили матрицу размера (20, 20).

Формы нужно привести к одному виду.

```

1 preds = preds.flatten()
2 print(preds.shape) # torch.Size([20])
3 print(y.shape)     # torch.Size([20])
4
5 diff = preds - y
6 print(diff.shape)  # torch.Size([20])

```

view меняет логическую форму, не копируя данные, если расположение тензора в памяти это допускает.

```

1 x_col = x.view(4, 1)
2 x_row = x.view(1, 4)
3 matrix = x_col + x_row

```

reshape удобнее, когда неизвестно, является ли память непрерывной: он возвращает представление, если может, а иначе создаёт копию.

5.4. IN-PLACE-ОПЕРАЦИИ

Методы с подчёркиванием в конце имени обычно изменяют существующий тензор.

```

1 y = x.exp() # Новый результат, x не меняется
2 x.exp_()    # x меняется на месте

```

Другие примеры: add_, zero_, relu_.

Один и тот же результат можно получить с разным числом временных буферов.

```

1 # Обычная запись создаёт промежуточные результаты
2 u = x + 2 * y - z
3
4 # Один новый рабочий буфер, затем его переиспользование
5 u = y.clone()
6 u.mul_(2)
7 u.add_(x)
8 u.sub_(z)

```

Математически результаты совпадают. Во втором варианте память `u` переиспользуется, но это допустимо лишь тогда, когда старое значение не понадобится для `backward`.

In-place-операции могут экономить память, но способны уничтожить значение, необходимое для `backward`. В таком случае PyTorch обычно останавливает вычисление с ошибкой. Особенно осторожно нужно изменять листовые тензоры, для которых требуется градиент.

5.5. УСТРОЙСТВО И ТИП ДАННЫХ

Устройство тензора можно проверить через `x.device`. Универсальный шаблон выбора CUDA выглядит так.

```

1 device = torch.device(
2     "cuda" if torch.cuda.is_available() else "cpu"
3 )
4
5 model = model.to(device)
6 x = x.to(device)
7
8 print(x.device)
9 new_tensor = torch.zeros(4, device=device)
10 cpu_tensor = x.cpu()

```

Методы `.cuda()` и `.cpu()` тоже существуют, однако `.to(device)` удобнее для кода, который должен работать и с GPU, и без него.

Модель и все тензоры одной операции должны находиться на совместимых устройствах. Частые копирования между CPU и GPU дороги, поэтому полный шаг обучения стараются выполнять на GPU, а на CPU переносить только готовые результаты и статистики.



Почему нужно замечать скрытые копирования

В ранних реализациях некоторых операций, например SVD, GPU-версия могла отсутствовать. Тогда данные незаметно для основной логики кода переносились на CPU, операция выполнялась там, а результат возвращался на GPU. Такие перемещения способны оказаться дороже самого вычисления.

Вещественные тензоры PyTorch по умолчанию обычно имеют тип `float32`, а NumPy обычно использует `float64`. На GPU часто применяются `float32`, `float16` и `bfloat16`. Повышенная точность может быть важна для чувствительных задач линейной алгебры на CPU, но требует больше памяти и вычислений.

6. Как работает autograd

6.1. REQUIRES_GRAD И BACKWARD

Чтобы PyTorch отслеживал операции над тензором, при его создании можно указать `requires_grad=True`.

```
1 x = torch.randn(4, requires_grad=True)
2 y = torch.randn(4, requires_grad=True)
3
4 z = x * y + 10
5 loss = z.sum()
6 loss.backward()
7
8 assert torch.allclose(x.grad, y)
9 assert torch.allclose(y.grad, x)
```

После `backward()` поле `.grad` листового тензора содержит вычисленный градиент. В этом примере $\nabla_x \mathcal{L} = y$ и $\nabla_y \mathcal{L} = x$; добавленная константа 10 на градиенты не влияет.

Обычный вызов `backward()` без аргумента предполагает скалярный выход. Для векторного $u = f(x)$ можно передать входящий градиент v .

```
1 u.backward(v)
```

Тогда PyTorch вычислит не полный якобиан, а векторно-якобианное произведение:

$$J_f(x)^T v.$$

Именно такие произведения последовательно вычисляет `reverse mode`.

6.2. FORWARD MODE И JVP

Автоматическое дифференцирование может вычислять производные проходом вперёд (*forward mode*) или проходом назад (*reverse mode*).

В `forward mode` вместе со значением x распространяется касательный вектор v . Для функции f результатом является произведение якобиана на вектор — JVP:

$$J_f(x)v.$$

В PyTorch его можно вычислить функцией `jvp`.

```

1 import torch
2 from torch.autograd.functional import jvp
3
4 def function(x):
5     return x ** 2
6
7 x = torch.tensor([2.0], requires_grad=True)
8 tangent = torch.tensor([1.0])
9
10 y, jvp_value = jvp(function, (x,), (tangent,))
11 print("Function value:", y)      # tensor([4.])
12 print("JVP value:", jvp_value)  # tensor([4.])

```

Forward AD может быть полезно, например, когда выходов больше, чем входов. В reverse mode от выхода распространяется входящий градиент v , а результатом является VJP:

$$J_f(x)^T v.$$

Для нейросетей с большим числом параметров и скалярной функцией потерь обычно используется reverse mode.

6.3. CLONE И DETACH

Вызов

```
1 y = x.clone()
```

создаёт отдельное хранилище, но сохраняет связь с графом. Градиент, пришедший через y , продолжит распространяться к x .

Вызов

```
1 y = x.detach()
```

разрывает связь с предыдущим графом: для autograd значение y становится константой.

При этом `detach()` обычно не копирует данные и может разделять память с исходным тензором.

Независимая копия без связи с графом создаётся так.

```
1 y = x.detach().clone()
```

Сравним три варианта явно.

```

1 x = torch.randn(3, requires_grad=True)
2
3 cloned = x.clone()           # Новая память, граф сохранён
4 detached = x.detach()        # Граф разорван, память обычно общая
5 independent = x.detach().clone() # Новая память, граф разорван

```

Изменять листовой тензор с `requires_grad=True` на месте нельзя.

```
1 y = torch.randn(4, requires_grad=True)
2 safe = y.clone().exp_() # Меняется промежуточный тензор
3 # y.exp_()              # RuntimeError: in-place для leaf-тензора
```

6.4. ГРАДИЕНТЫ НАКАПЛИВАЮТСЯ

Новый результат backward **прибавляется** к уже существующему `.grad`.

```
1 loss_1.backward()
2 loss_2.backward()
3 # parameter.grad = grad(loss_1) + grad(loss_2)
```

Это необходимо, если один параметр используется в нескольких ветвях графа. Например, общая матрица W получает сумму вкладов по всем путям:

$$\nabla_W \mathcal{L} = \sum_j \frac{\partial \mathcal{L}}{\partial W} \Big|_j.$$

Такая ситуация типична для рекуррентной сети, где одна матрица W_{hh} используется на всех шагах:

$$h_t = \varphi(W_{hh}h_{t-1} + W_{xh}x_t + b).$$

Между независимыми шагами оптимизации старые градиенты обычно не нужны, поэтому перед новым мини-батчем их обнуляют.

```
1 optimizer.zero_grad()
```

Осознанное накопление через несколько мини-батчей возможно, но это уже отдельный приём.

Накопление легко увидеть на числах.

```
1 x = torch.ones(4, requires_grad=True)
2
3 (x.square().sum()).backward()
4 print(x.grad) # tensor([2., 2., 2., 2.])
5
6 ((1 / x).sum()).backward()
7 print(x.grad) # tensor([1., 1., 1., 1.]): 2 + (-1)
8
9 x.grad.zero_()
10 ((1 / x).sum()).backward()
11 print(x.grad) # tensor([-1., -1., -1., -1.])
```

6.5. ЛИСТОВЫЕ И ПРОМЕЖУТОЧНЫЕ ТЕНЗОРЫ

```

1  x = torch.randn(4, requires_grad=True) # Листовая вершина
2  z = torch.randn(4, requires_grad=True) # Листовая вершина
3  w = 2 * z                               # Промежуточная вершина
4  y = x * w + 1
5  loss = y.sum()
6
7  w.retain_grad() # Запросить градиент промежуточной вершины
8  loss.backward()
9
10 print(x.grad) # dL/dx = w
11 print(z.grad) # dL/dz = 2x
12 print(w.grad) # dL/dw = x

```

По умолчанию PyTorch сохраняет `.grad` у созданных пользователем листовых тензоров. В примере это `x` и `z`. Тензор `w = 2z` является промежуточной вершиной: его градиент участвует в обратном проходе, но после него не хранится. Метод `retain_grad()` просит сохранить этот градиент явно. Вызывать его нужно **до** `backward()`.

6.6. ЖИЗНЕННЫЙ ЦИКЛ ГРАФА

После обычного `backward()` сохранённые промежуточные данные графа освобождаются. Повторный `backward` по тому же графу требует явного сохранения.

```

1  loss.backward(retain_graph=True)

```

Этот флаг увеличивает расход памяти и нужен редко. Для производных высших порядков важно не только сохранить старый граф, но и построить граф вычисления самого градиента.

```

1  grad = torch.autograd.grad(
2      loss,
3      parameters,
4      create_graph=True,
5  )

```

Например, так можно вычислять произведение гессиана на вектор:

$$H(\theta)v = \nabla_{\theta}(\nabla_{\theta}\mathcal{L}(\theta)^T v).$$

6.7. ОБУЧЕНИЕ И ИНФЕРЕНС — РАЗНЫЕ РЕЖИМЫ

Нужно различать две независимые настройки.

Команда	Что она делает
<code>model.train()</code>	Включает обучающее поведение Dropout, BatchNorm и других режим-зависимых слоёв
<code>model.eval()</code>	Включает их поведение для оценки и инференса
<code>torch.no_grad()</code>	Отключает построение графа autograd
<code>torch.inference_mode()</code>	Более строгий и экономный режим чистого инференса

Таблица 4. Команды, управляющие режимом модели и построением графа



`model.eval()` **не отключает градиенты**

При оценке модели обычно одновременно вызывают `model.eval()` и выполняют вычисления внутри `with torch.no_grad():`. Первая команда меняет поведение некоторых слоёв, вторая — действительно отключает построение графа.

Для функции инференса тот же режим можно задать декоратором.

```

1 @torch.no_grad()
2 def predict(model, x):
3     return model(x)
```

7. Dataset и DataLoader

Большая выборка может не помещаться в память целиком, поэтому данные читают и обрабатывают мини-батчами.

7.1. DATASET

Объект `Dataset` определяет,

- где лежат данные, сколько в выборке объектов;
- как по индексу получить один объект и его метку.

Собственный датасет обычно реализует `__len__` и `__getitem__`.

```

1 from torch.utils.data import Dataset
2
3 class MyDataset(Dataset):
4     def __init__(self, items, transform=None):
5         self.items = items
6         self.transform = transform
7
8     def __len__(self):
9         return len(self.items)
10
11    def __getitem__(self, index):
12        x, y = load_item(self.items[index])
13        if self.transform is not None:
14            x = self.transform(x)
15        return x, y

```

`load_item` здесь обозначает пользовательскую процедуру чтения. Она может загружать объект с диска только в момент обращения по индексу.

7.2. DATALOADER

`DataLoader` создаёт итератор по мини-батчам.

```

1 from torch.utils.data import DataLoader
2
3 train_loader = DataLoader(
4     train_dataset,
5     batch_size=64,
6     shuffle=True,
7     num_workers=4,
8 )

```

Он отвечает за формирование батчей, перемешивание объектов, параллельную загрузку, предварительную подготовку следующих батчей.

Одна эпоха обычно соответствует одному полному проходу по `train_loader`.

Если признаки и ответы уже находятся в NumPy-массивах или тензорах, удобно использовать готовый `TensorDataset`.

```

1 import torch
2 from torch.utils.data import TensorDataset, DataLoader
3
4 def make_loader(X, y, batch_size=64, shuffle=False):
5     dataset = TensorDataset(
6         torch.as_tensor(X, dtype=torch.float32),
7         torch.as_tensor(y, dtype=torch.long),
8     )
9     return DataLoader(
10        dataset,

```

```

11         batch_size=batch_size,
12         shuffle=shuffle,
13     )
14
15 train_loader = make_loader(X_train, y_train, shuffle=True)
16 val_loader = make_loader(X_val, y_val)
17 test_loader = make_loader(X_test, y_test)

```

Для обучения объекты обычно перемешивают, а для валидации и теста это не требуется. Деление на train, validation и test выполняют **до** создания загрузчиков; способ разбиения должен учитывать структуру задачи и не допускать утечки данных.

7.3. ПРЕОБРАЗОВАНИЯ ДАННЫХ

Данные удобно преобразовывать во время загрузки. В частности, можно масштабировать и нормализовать:

$$x' = \frac{x - \mu}{\sigma}.$$

Обычно transform является частью Dataset, а не DataLoader.

8. Модель как nn.Module

Нейросеть обычно оформляется как класс, наследующий nn.Module. Минимально нужно определить конструктор и прямой проход.

```

1 import torch.nn as nn
2
3 class MLP(nn.Module):
4     def __init__(self, input_dim, hidden_dim, n_classes):
5         super().__init__()
6         self.layers = nn.Sequential(
7             nn.Linear(input_dim, hidden_dim),
8             nn.ReLU(),
9             nn.Linear(hidden_dim, n_classes),
10        )
11
12    def forward(self, x):
13        return self.layers(x)

```

super().__init__() необходим, чтобы PyTorch зарегистрировал вложенные слои и их параметры. После этого вызов model(x) автоматически запускает forward.

Основным преимуществом nn.Module является интерфейс работы с параметрами, буферами и подмодулями. Если параметр, буфер или подмодуль зарегистрирован, становятся доступны методы:

- parameters() и named_parameters();
- buffers() и named_buffers();

- `requires_grad_()`;
- `to(device)` и `to(dtype)`;
- `train()` и `eval()`.

Есть два способа регистрации.

1. **Неявная регистрация.** Если атрибуту модуля присвоить `torch.nn.Parameter` или другой `nn.Module`, объект будет зарегистрирован автоматически. Обычный тензор при присваивании атрибуту параметром не становится; неявной регистрации буферов нет.
2. **Явная регистрация.** Для неё используются `register_parameter`, `register_buffer` и `register_module`.

В большинстве случаев предпочтительна неявная регистрация.

```

1 class LinearModel(nn.Module):
2     def __init__(self, input_dim, output_dim):
3         super().__init__()
4         self.weight = nn.Parameter(
5             torch.randn(output_dim, input_dim)
6         )
7         self.bias = nn.Parameter(torch.zeros(output_dim))
8
9     def forward(self, x):
10        return x @ self.weight.T + self.bias

```

Обычные списки и словари Python автоматически не регистрируют помещённые в них параметры и модули. Для этого используются специальные контейнеры:

- `nn.ParameterList` и `nn.ParameterDict` — для параметров;
- `nn.ModuleList` и `nn.ModuleDict` — для модулей.

`nn.ModuleList` и `nn.ModuleDict` только хранят зарегистрированные модули: порядок их применения задаётся в `forward`. `nn.Sequential`, наоборот, сам последовательно применяет вложенные модули к входу.

Готовые строительные блоки включают:

- `nn.Linear`;
- `nn.ReLU`;
- `nn.Sequential`.

8.1. ИЕРАРХИЯ БЛОКОВ

`nn.Module` может содержать другие модули, поэтому модель естественно строится как иерархия:

- простые операции объединяются в слой;
- слои — в функциональный блок;
- блоки — в полную модель.

Такое устройство упрощает переиспользование, замену и сохранение частей сети.

8.2. ПРЕДОБУЧЕНИЕ И НОВАЯ «ГОЛОВА»

Модель удобно представить как композицию

$$f(x) = h_{\psi}(g_{\phi}(x)),$$

где g_{ϕ} извлекает общие признаки, а h_{ψ} решает конкретную задачу, например классификацию.

Если целевая выборка мала:

- 1) модель предварительно обучают на большом наборе данных;
- 2) исходную классификационную голову удаляют;
- 3) извлекатель признаков сохраняют;
- 4) добавляют новую голову;
- 5) обучают новую часть или дообучают всю модель на целевой выборке.

Это один из базовых сценариев переноса обучения (*transfer learning*).

8.3. ЧИСЛО ПАРАМЕТРОВ

Параметры зарегистрированных модулей доступны через `model.parameters()`.

```

1 n_parameters = sum(
2     parameter.numel()
3     for parameter in model.parameters()
4 )
5
6 n_trainable = sum(
7     parameter.numel()
8     for parameter in model.parameters()
9     if parameter.requires_grad
10 )

```

После создания модели полезно сразу проверить порядок числа параметров и сопоставить ёмкость сети с объёмом данных. Миллионы параметров при небольшой выборке могут сделать обучение с нуля неустойчивым и привести к переобучению.



Это эвристика, а не строгий закон

Число параметров не обязано быть близко к числу обучающих объектов. Большие предобученные и перепараметризованные модели могут хорошо работать и на малых целевых выборках. Важно учитывать предобучение, регуляризацию, структуру модели и сложность задачи.

9. Оптимизатор и цикл обучения

Сначала модель один раз переносят на целевое устройство, затем создают оптимизатор. Оптимизатор получает ссылки именно на размещённые там параметры модели.

```

1 model = MLP(input_dim, hidden_dim, n_classes).to(device)
2
3 optimizer = torch.optim.Adam(
4     model.parameters(),
5     lr=1e-3,
6 )

```

После вычисления градиентов `optimizer.step()` обновляет эти параметры. Для обычного градиентного спуска правило имело бы вид

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}.$$

Adam использует более сложное адаптивное обновление, но внешний интерфейс цикла остаётся тем же.

9.1. ОДНА ЭПОХА ОБУЧЕНИЯ

```

1 def train_epoch(model, loader, criterion, optimizer, device):
2     model.train()
3
4     total_loss = 0.0
5     n_objects = 0
6
7     for x, y in loader:
8         x = x.to(device)
9         y = y.to(device)
10
11         optimizer.zero_grad()
12
13         prediction = model(x)
14         loss = criterion(prediction, y)
15
16         loss.backward()
17         optimizer.step()
18
19         batch_size = x.size(0)
20         total_loss += loss.item() * batch_size
21         n_objects += batch_size
22
23     return total_loss / n_objects

```

Последовательность одного шага:

- 1) загрузить мини-батч и перенести его на нужное устройство;
- 2) обнулить градиенты предыдущего шага;
- 3) выполнить прямой проход;
- 4) вычислить скалярную функцию потерь;
- 5) вызвать `loss.backward()`;
- 6) обновить параметры через `optimizer.step()`.

После `backward()` градиенты находятся в полях `.grad` параметров, возвращаемых `model.parameters()`.

10. Оценка модели

Во время валидации и тестирования параметры не обновляются, а граф производных обычно не нужен.

```

1  def evaluate(model, loader, criterion, device):
2      model.eval()
3
4      total_loss = 0.0
5      n_objects = 0
6      n_correct = 0
7
8      with torch.no_grad():
9          for x, y in loader:
10             x = x.to(device)
11             y = y.to(device)
12
13             prediction = model(x)
14             loss = criterion(prediction, y)
15
16             batch_size = x.size(0)
17             total_loss += loss.item() * batch_size
18             n_objects += batch_size
19
20             predicted_class = prediction.argmax(dim=1)
21             n_correct += (predicted_class == y).sum().item()
22
23     return {
24         "loss": total_loss / n_objects,
25         "accuracy": n_correct / n_objects,
26     }

```

Функция потерь возвращает нульмерный тензор. Метод `.item()` извлекает из него обычное Python-число, удобное для логирования и накопления статистик.

Если функция потерь усредняет значения внутри батча, при вычислении среднего по всей выборке нужно умножить `loss.item()` на фактический размер батча. Это особенно важно для последнего неполного батча.

11. Основные выводы

1. Backpropagation — это reverse-mode automatic differentiation по вычислительному графу.
2. Дифференциалы позволяют получать матричные backward-формулы без покомпонентных индексов.
3. Для последнего слоя с softmax и кросс-энтропией $\delta^{(L)} = p - e_y$. Для скрытого слоя $\ell = 1, \dots, L - 1$

$$\delta^{(\ell)} = \nabla_{a^{(\ell)}} \mathcal{L} \odot \phi'_{\ell}(z^{(\ell)}),$$

$$\nabla_{W^{(\ell)}} \mathcal{L} = \delta^{(\ell)} (a^{(\ell-1)})^T, \quad \nabla_{b^{(\ell)}} \mathcal{L} = \delta^{(\ell)}, \quad \nabla_{a^{(\ell-1)}} \mathcal{L} = (W^{(\ell)})^T \delta^{(\ell)}.$$

4. Дифференцирование через `solve(A, b)` сводится к решению системы с A^T и внешнему произведению.
5. PyTorch сочетает тензорные вычисления, ускорители, динамический граф и автоматическое дифференцирование.
6. `.grad` накапливается, поэтому перед обычным шагом оптимизации градиенты обнуляют.
7. `model.train()` и `model.eval()` управляют поведением слоёв, а `torch.no_grad()` — построением вычислительного графа.
8. `Dataset` описывает получение отдельного объекта, `DataLoader` организует мини-батчи, а `nn.Module` задаёт иерархию модели.
9. Базовый шаг обучения имеет вид `zero_grad` → `forward` → `loss` → `backward` → `step`.



Ключевые формулы

Покомпонентная активация

$$\nabla_z \mathcal{L} = \nabla_a \mathcal{L} \odot \phi'(z).$$

Softmax и кросс-энтропия

$$\nabla_{z^{(L)}} \mathcal{L} = p - e_y.$$

Аффинный слой $z = Wa + b$

$$\nabla_W \mathcal{L} = (\nabla_z \mathcal{L}) a^T, \quad \nabla_b \mathcal{L} = \nabla_z \mathcal{L}, \quad \nabla_a \mathcal{L} = W^T \nabla_z \mathcal{L}.$$

Решение $Ax = b$

$$A^T \lambda = \nabla_x \mathcal{L}, \quad \nabla_b \mathcal{L} = \lambda, \quad \nabla_A \mathcal{L} = -\lambda x^T.$$